

École Supérieure Privée d'Ingénierie et de Technologies — ESPRIT

RAPPORT DE STAGE D'IMMERSION EN ENTREPRISE

SOTETEL AI-NIDS

Plateforme de détection d'intrusions réseau assistée
par intelligence artificielle

Élaboré par :
Omar Belhaj

Encadrant :
Imed Jlailia

Organisme d'accueil : SOTETEL

Année universitaire : 2025-2026

Contexte du projet : cybersécurité / détection d'intrusions réseau assistée par IA

Table des matières

Introduction générale	3
Chapitre 1 - Cadre général du projet	4
1.1 Organisme d'accueil et cadre du stage	4
1.2 Périmètre et objectifs du projet	4
1.3 Étude de l'existant et besoins identifiés	4
1.4 Solution proposée	5
1.5 Méthodologie de travail	5
Chapitre 2 - Analyse et spécification des besoins	6
2.1 Identification des acteurs	6
2.2 Besoins fonctionnels	6
2.3 Besoins non fonctionnels	7
2.4 Cas d'utilisation central - Détecter et persister une alerte d'intrusion en temps réel	7
2.5 Diagramme général de cas d'utilisation	8
Chapitre 3 - Conception	9
3.1 Architecture générale du système	9
3.2 Conception du laboratoire réseau	10
3.3 Conception de la persistance des données	11
3.3.1 Diagramme conceptuel de classes	11
3.4 Conception de la sécurité et des frontières de confiance	12
3.5 Principes de conception des interfaces	12
Chapitre 4 - Réalisation	13
4.1 Environnement de travail et technologies	13
4.2 Détecteur Ubuntu en temps réel	13
4.3 Backend FastAPI et opérations API	16
4.4 Tableau de bord de supervision React	16
4.5 Persistance PostgreSQL et hygiène des données	19
4.6 Mise en œuvre de la sécurité applicative	20
Chapitre 5 - Intelligence artificielle et détection d'intrusions	23
5.1 Capture des données et extraction des caractéristiques	23
5.2 Modèle de base au niveau paquet	23
5.3 Modèle comportemental à cinq classes et validation	24
5.4 Interprétation des résultats	27
Chapitre 6 - DevSecOps, déploiement et validation	28
6.1 Docker et déploiement reproductible	28
6.2 Intégration continue avec GitHub Actions	28
6.3 Validation de bout en bout et preuves d'ingénierie	29
6.4 Principales leçons de débogage et de récupération	30
6.5 Limites actuelles	30
6.6 Perspectives	31
Conclusion générale	32
Annexes	33

Liste des figures

Figure 1 - Diagramme général de cas d'utilisation.....	8
Figure 2 - Architecture logique finale	9
Figure 3 - Topologie du laboratoire EVE-NG.....	10
Figure 4 - Diagramme conceptuel de classes.....	12
Figure 5 - État systemd du détecteur Ubuntu.....	13
Figure 6 - JSON d'état de santé en exécution	14
Figure 7 - Journal structuré du détecteur temps réel	15
Figure 8 - Interface sécurisée de connexion.....	17
Figure 9 - Tableau de bord de supervision administrateur.....	18
Figure 10 - Vue détaillée d'une alerte.....	19
Figure 11 - Alertes comportementales persistées	19
Figure 12 - Chemins d'authentification humain et machine	20
Figure 13 - Preuve du journal d'audit PostgreSQL	21
Figure 14 - Santé API et en-têtes de sécurité HTTP	22
Figure 15 - Évaluation du modèle paquet de base	24
Figure 16 - Évaluation du modèle comportemental à cinq classes	25
Figure 17 - Résumé du holdout contrôlé par session	26
Figure 18 - Exécution CI GitHub Actions.....	29

Introduction générale

Contexte

Les infrastructures réseau sont continuellement exposées à des activités de reconnaissance, à des abus d'authentification et à des comportements de déni de service. La supervision conventionnelle reste utile, mais l'augmentation du volume et de la diversité du trafic rend l'analyse manuelle difficile. Dans ce contexte, l'apprentissage automatique peut compléter les mécanismes de détection en identifiant automatiquement des comportements suspects à partir de caractéristiques observées sur le trafic réseau.

Problématique

La problématique centrale du projet consiste à détecter des comportements réseau suspects en quasi temps réel tout en conservant une chaîne de traitement compréhensible, reproductible et sécurisée. Le système doit distinguer le trafic normal de plusieurs catégories d'attaques, limiter les fausses alertes opérationnelles, authentifier les acteurs humains et le capteur machine, conserver une trace des événements sensibles et présenter les résultats dans une interface exploitable.

Objectifs

- Construire un laboratoire EVE-NG reproductible pour générer du trafic normal et du trafic malveillant borné.
- Capturer le trafic PCAP et le transformer en caractéristiques au niveau paquet et en caractéristiques comportementales causales.
- Détecter le trafic normal et quatre catégories d'attaques : scan de ports, force brute, SYN flood et ICMP flood.
- Exploiter un détecteur Ubuntu en temps réel avec seuils de confiance, délai anti-duplication, état de santé et démarrage par systemd.
- Persister les alertes authentifiées via FastAPI et PostgreSQL, puis les visualiser dans un tableau de bord React.
- Séparer l'authentification humaine JWT/RBAC de l'authentification machine du capteur.
- Conteneuriser les services applicatifs et les valider automatiquement avec GitHub Actions CI.

Organisation du rapport

Ce rapport est organisé en six chapitres. Le premier présente le cadre général du projet et son périmètre. Le deuxième formalise les besoins fonctionnels et non fonctionnels. Le troisième décrit la conception de l'architecture, du laboratoire réseau, de la persistance et des frontières de confiance. Le quatrième présente la réalisation technique. Le cinquième est consacré au module d'intelligence artificielle et à l'évaluation de la détection. Enfin, le sixième traite du déploiement, de l'intégration continue, de la validation de bout en bout, des difficultés rencontrées et des perspectives.

Chapitre 1 - Cadre général du projet

1.1 Organisme d'accueil et cadre du stage

Le projet a été réalisé dans le cadre d'un stage au sein de SOTETEL. Les éléments de preuve du projet identifient SOTETEL comme organisme d'accueil et inscrivent le travail dans un contexte de cybersécurité, d'intelligence artificielle et de détection d'intrusions réseau. Le développement a été mené comme un prototype de laboratoire complet, depuis la génération de trafic jusqu'à la visualisation des alertes détectées.

1.2 Périmètre et objectifs du projet

L'objectif du projet était de concevoir et d'implémenter un NIDS pratique assisté par IA, démontrable de bout en bout dans un laboratoire réseau virtuel isolé, tout en restant compréhensible, reproductible et suffisamment sécurisé pour constituer un livrable de stage.

- Construire un environnement EVE-NG pour générer de manière reproductible du trafic normal et malveillant.
- Capturer le trafic réseau au format PCAP et le convertir en caractéristiques exploitables par l'apprentissage automatique.
- Conserver un classifieur simple au niveau paquet tout en développant un modèle comportemental causal plus adapté à la détection en temps réel.
- Détecter le trafic normal et quatre catégories d'attaques : scan de ports, force brute, SYN flood et ICMP flood.
- Créer un détecteur Ubuntu en temps réel avec seuils de confiance, délais anti-duplication, journaux rotatifs, état de santé et démarrage systemd.
- Persister les alertes via FastAPI et PostgreSQL, puis les visualiser dans un tableau de bord React.
- Séparer l'authentification humaine (JWT/RBAC) de l'authentification machine (clé API du capteur).
- Conditionner la pile applicative avec Docker Compose et la valider automatiquement avec GitHub Actions CI.

1.2.1 Limites du périmètre

- La solution est validée comme prototype de laboratoire et non comme IDS de production.
- La génération d'attaques est bornée et autorisée dans l'environnement EVE-NG.
- Le capteur Ubuntu en temps réel reste externe à Docker Compose ; Docker ne conditionne que les services applicatifs.
- HTTPS/TLS et une infrastructure de gestion de secrets de production ne sont pas déployés dans le MVP actuel.

1.3 Étude de l'existant et besoins identifiés

Le projet est parti d'un prototype hors ligne de classification de paquets. Cette base permettait de classer des échantillons contrôlés, mais ne fournissait ni contexte temporel, ni capture en temps réel, ni persistance sécurisée, ni gestion des rôles, ni audit, ni chaîne de déploiement reproductible. Ces limites ont conduit à faire évoluer la solution vers une plateforme NIDS complète intégrant laboratoire réseau, modèle comportemental, capteur Linux, API sécurisée, base PostgreSQL et tableau de bord.

1.4 Solution proposée

La solution proposée est un NIDS de laboratoire assisté par IA composé d'un environnement de trafic EVE-NG autorisé, d'un capteur Ubuntu en temps réel, d'un modèle Random Forest comportemental à

cinq classes, d'un backend FastAPI, d'une base PostgreSQL et d'un tableau de bord React. Les services applicatifs sont conteneurisés avec Docker Compose et leur construction est validée par GitHub Actions.

1.5 Méthodologie de travail

Le développement et la validation ont été réalisés de manière incrémentale. Chaque sous-système a été testé indépendamment avant son intégration de bout en bout : capture et extraction des caractéristiques, entraînement du modèle, détection temps réel, API, base de données, tableau de bord, sécurité, conteneurisation et intégration continue. Une fois un jalon validé, les tests ont été répétés uniquement lorsque des modifications susceptibles de l'affecter étaient introduites.

Chapitre 2 - Analyse et spécification des besoins

2.1 Identification des acteurs

Acteur	Rôle dans le système
Lecteur	S'authentifie sur le tableau de bord et consulte les alertes, les filtres, les statistiques et le détail des alertes.
Analyste	Dispose des droits du lecteur et peut utiliser les opérations protégées de prédiction/analyse.
Administrateur	Dispose du rôle humain le plus élevé, incluant la suppression privilégiée et l'accès aux journaux d'audit.
Capteur NIDS Ubuntu	Acteur machine qui capture le trafic, réalise l'inférence comportementale et transmet les alertes retenues avec X-NIDS-API-Key.
Générateur de trafic contrôlé	Nœud Kali utilisé uniquement dans le laboratoire EVE-NG autorisé pour générer du trafic normal et des scénarios d'attaque bornés.

2.2 Besoins fonctionnels

Module	Principales exigences fonctionnelles
Authentification et autorisation	Connexion utilisateur, émission de JWT, point de terminaison de l'utilisateur courant, RBAC hiérarchique lecteur/analyste/administrateur et authentification séparée du capteur.
Détection en temps réel	Capter les paquets sur ens3, construire des échantillons de 5 secondes avec un contexte causal de 30 secondes, classer le trafic, appliquer les règles de warm-up/seuil/cooldown et émettre les alertes retenues.
Gestion des alertes	Persister les alertes temps réel, lister/filtrer/paginer les alertes, consulter leur détail, calculer les statistiques et autoriser la suppression uniquement à l'administrateur.
Tableau de bord de supervision	Afficher les cartes statistiques, graphiques d'attaque/sévérité/protocole, alertes récentes, filtres, pagination, export CSV, état de santé et actualisation automatique.
Audit	Persister les succès/échecs d'authentification, refus d'autorisation, échecs du capteur et événements de sécurité administratifs ; exposer les journaux d'audit aux administrateurs.
Services d'apprentissage automatique	Fournir la prédiction du modèle paquet et utiliser le Random Forest comportemental à cinq classes pour le détecteur temps réel.
Déploiement et CI	Exécuter les services applicatifs de manière reproductible avec Docker Compose et valider les builds backend, frontend et images Docker via GitHub Actions.

2.3 Besoins non fonctionnels

Catégorie	Exigence
Sécurité	Hachage Argon2 des mots de passe, jetons JWT Bearer, RBAC appliqué côté backend, clé capteur séparée, liste blanche CORS, limitation de débit et en-têtes de sécurité HTTP.

Catégorie	Exigence
Traçabilité	Persister les actions sensibles et les preuves du détecteur afin de permettre l'audit des authentifications et actions administratives.
Fiabilité	Utiliser un état de santé, des journaux rotatifs, le démarrage/redémarrage systemd et des contrôles de santé API/base/modèle.
Reproductibilité	Utiliser des scénarios EVE-NG contrôlés, Docker Compose, une configuration par variables d'environnement et une validation CI automatisée.
Maintenabilité	Séparer les responsabilités capteur, backend, frontend, persistance et ML ; conserver la configuration dans les variables d'environnement plutôt que dans le code source.
Performance / exploitabilité	Utiliser des fenêtres d'observation courtes, des seuils de confiance et des règles de cooldown pour limiter les alertes bruyantes ou dupliquées.
Évaluation responsable	Considérer les métriques de laboratoire comme des preuves contrôlées et non comme des garanties de performance IDS dans tout environnement réel.

2.4 Cas d'utilisation central - Détecter et persister une alerte d'intrusion en temps réel

Élément	Description
Acteur principal	Capteur NIDS Ubuntu
Préconditions	Le service capteur est actif, le modèle est chargé, l'interface surveillée est disponible et la clé capteur du backend est configurée.
Déclencheur	Du trafic est observé sur l'interface EVE-NG surveillée.
Scénario nominal	1) capturer les paquets ; 2) construire les caractéristiques comportementales causales ; 3) prédire la classe/confiance ; 4) supprimer les événements normaux, peu confiants ou dupliqués ; 5) s'authentifier auprès de FastAPI ; 6) persister l'alerte dans PostgreSQL ; 7) l'exposer dans le tableau de bord React.
Scénario alternatif	Si le chemin de gestion/API est indisponible, l'état de santé et les journaux du détecteur signalent l'échec ; si la confiance est sous le seuil, la prédiction est volontairement supprimée.
Postcondition	Un événement retenu est stocké et devient visible via les endpoints protégés d'alertes/statistiques et le tableau de bord.

2.5 Diagramme général de cas d'utilisation

Le diagramme général de cas d'utilisation résume les interactions entre les trois rôles humains (Lecteur, Analyste et Administrateur) et le capteur AI-NIDS. Il met en évidence l'accès authentifié au tableau de bord et aux alertes, l'analyse protégée, les opérations réservées à l'administrateur et l'envoi automatique d'alertes par le capteur.

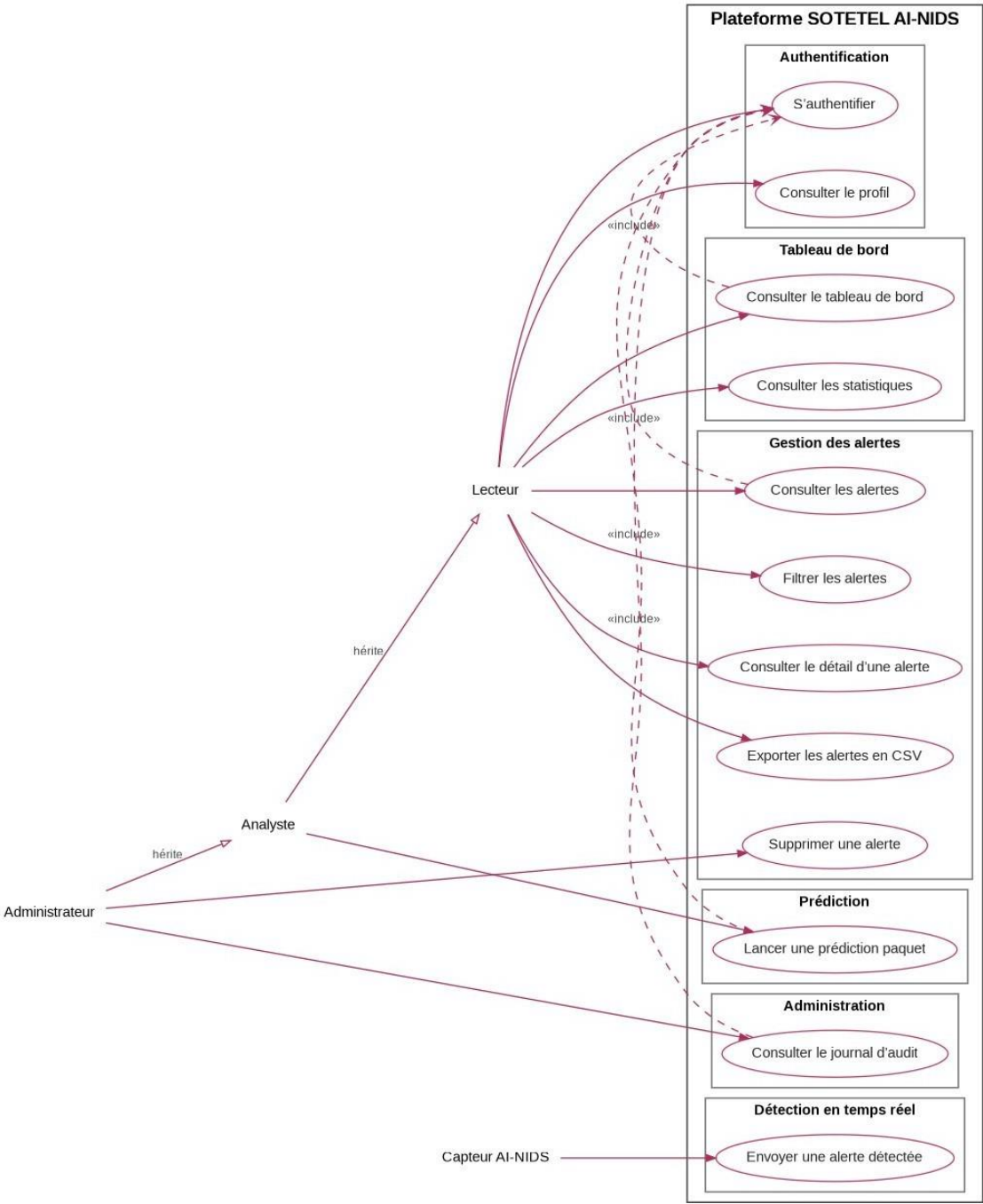


Figure 1 - Diagramme général de cas d'utilisation de la plateforme SOTETEL AI-NIDS.

Chapitre 3 - Conception

3.1 Architecture générale du système

SOTETEL AI-NIDS - Final Logical Architecture

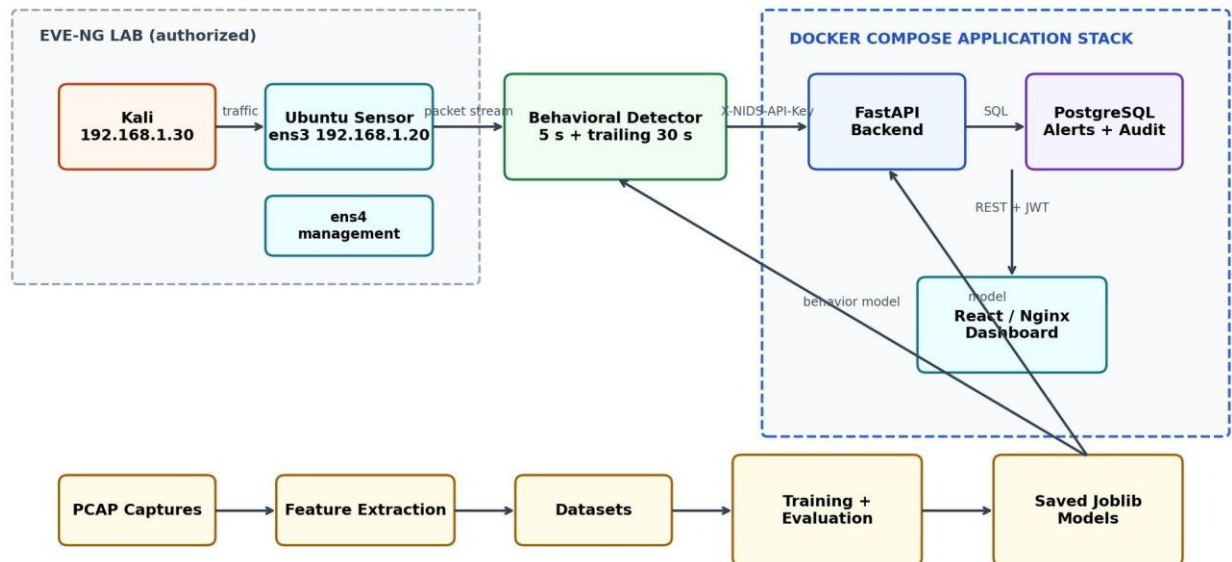


Figure 2 - Architecture logique finale : chemin du capteur EVE-NG, pile applicative et pipeline ML hors ligne.

L'architecture sépare volontairement trois préoccupations : le laboratoire de trafic autorisé, le chemin d'inférence de l'apprentissage automatique et les services applicatifs. Le capteur Ubuntu observe le trafic du laboratoire et n'envoie à FastAPI que les alertes répondant aux règles opérationnelles. Les opérateurs humains interagissent uniquement avec le chemin API/tableau de bord ; PostgreSQL n'est jamais accessible directement depuis le navigateur.

3.1.1 Frontières de confiance et réseau

Frontière	Composant	Rôle
Trafic du laboratoire	Kali -> Ubuntu ens3	Génère du trafic normal et des attaques bornées pour des tests autorisés.
Chemin de gestion	Ubuntu ens4 -> Windows/Backend	Transporte les contrôles de santé et la livraison authentifiée des alertes.
API applicative	React -> FastAPI	Opérations humaines protégées par JWT et données du tableau de bord.
Persistance	FastAPI -> PostgreSQL	Stocke les alertes, utilisateurs et enregistrements d'audit.
Build / déploiement	Docker Compose	Conditionne les services frontend, backend et base de données.
CI	GitHub Actions	Valide les builds backend, frontend et images Docker lors des modifications du dépôt.

3.2 Conception du laboratoire réseau

EVE-NG fournit l'environnement réseau isolé utilisé pour générer un trafic reproductible. Kali joue le rôle de nœud de génération de trafic contrôlé et Ubuntu joue à la fois le rôle de cible et de capteur AI-NIDS.

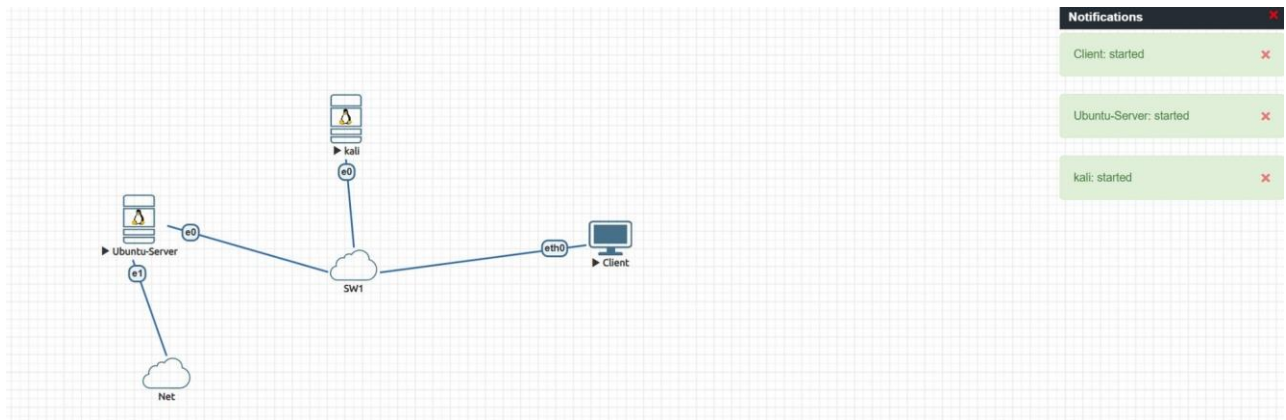


Figure 3 - Topologie finale du laboratoire EVE-NG avec Kali, le serveur Ubuntu capteur/cible, le nœud client et les liaisons réseau.

La topologie constitue un environnement de test isolé et autorisé utilisé pour produire du trafic normal ainsi que du trafic malveillant borné. Kali est le générateur de trafic contrôlé et Ubuntu Server héberge le capteur en temps réel.

Nœud / Interface	Adresse / Mode	Objectif
Kali eth0	192.168.1.30/24	Générateur de trafic autorisé et nœud attaquant.
Ubuntu ens3	192.168.1.20/24	Interface de laboratoire surveillée ; l'observation des paquets s'effectue ici.
Ubuntu ens4	192.168.182.x/24 via DHCP	Connectivité de gestion/API vers l'hôte backend Windows.
Windows VMnet8	192.168.182.1	Chemin hôte vers FastAPI lors des démonstrations EVE-NG natives.

3.2.1 Scénarios de trafic validés

Catégorie	Exemples	Objectif
Normal	ICMP ping, SSH, HTTP, UDP	Apprendre les modèles de communication normaux et réduire le biais vers les seules attaques.
Reconnaissance	TCP port scan	Créer une diversité de ports de destination et des rafales de connexions sur fenêtre courte.
Abus d'authentification	Comportement de force brute SSH	Créer des tentatives répétées de connexion/authentification depuis une source.
Déni de service	SYN flood	Créer une concentration SYN anormale et un débit de connexions élevé.
Déni de service	ICMP flood	Créer un comportement de requêtes echo à haut débit distinct d'un ping normal.

Principe de sécurité

Toutes les commandes d'attaque utilisées en démonstration sont bornées par le nombre de paquets, le débit, la plage de ports, le nombre de tâches ou une petite liste de mots de passe contrôlée. Les modes de flood non bornés sont volontairement évités.

3.3 Conception de la persistance des données

La couche de persistance repose sur PostgreSQL et stocke les alertes, les utilisateurs et les événements d'audit. La conception sépare les comptes humains, les événements réseau détectés et les traces de sécurité.

Zone de données	Objectif
alerts	Événements d'attaque détectés avec sévérité, attributs réseau, confiance, description et horodatage.
users	Nom d'utilisateur, hash Argon2 du mot de passe, rôle et statut actif.
audit_logs	Événements de sécurité d'authentification, d'autorisation, d'authentification du capteur et d'administration.

3.3.1 Diagramme conceptuel de classes

Le diagramme conceptuel de classes représente les principales entités applicatives et leurs relations. Les comptes utilisateurs sont associés à des rôles et génèrent des événements d'audit, le capteur en temps réel produit des alertes grâce au modèle de détection, et chaque alerte contient les informations réseau et le niveau de confiance nécessaires à son exploitation.

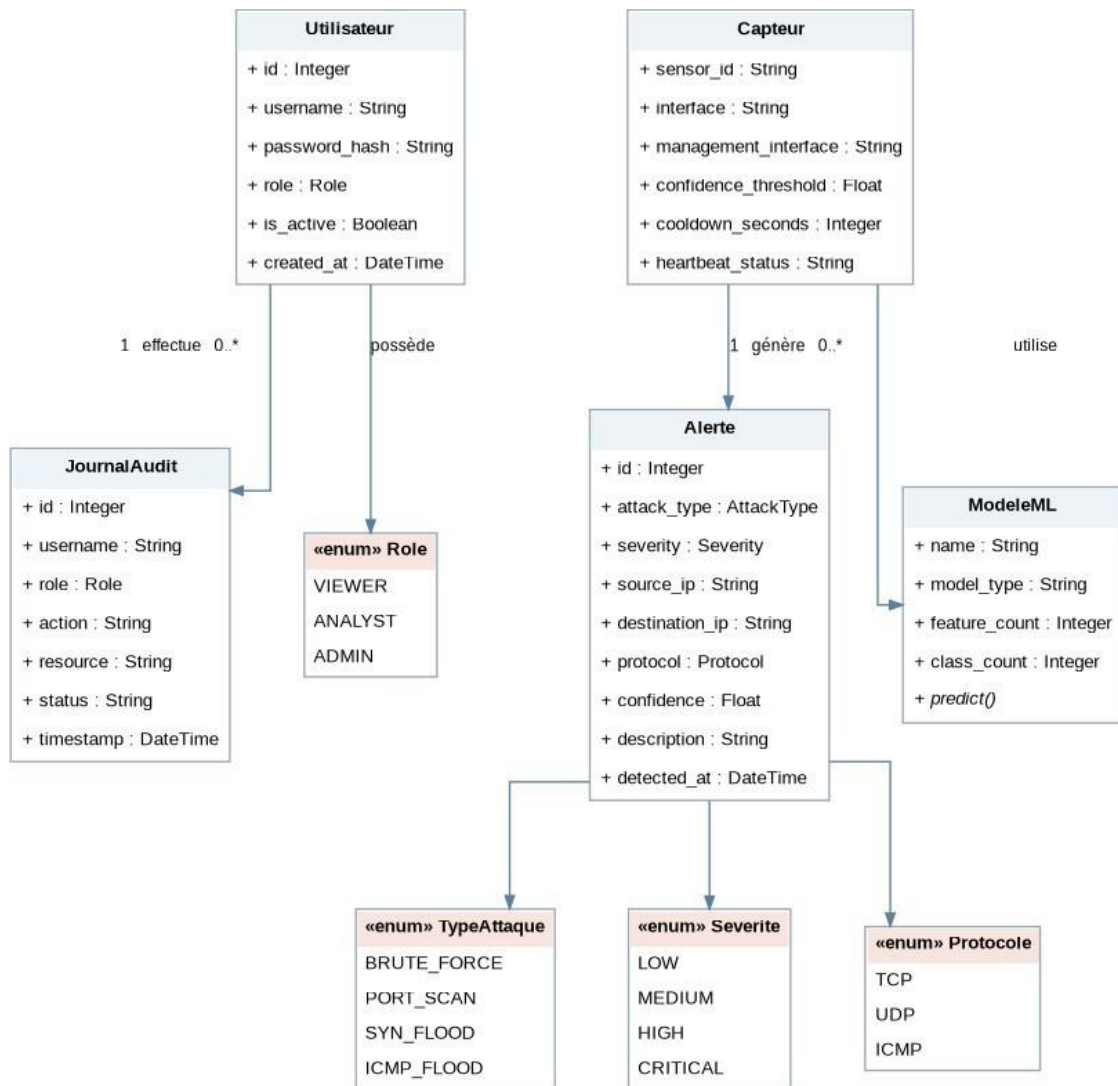


Figure 4 - Diagramme conceptuel de classes de la plateforme SOTETEL AI-NIDS.

3.4 Conception de la sécurité et des frontières de confiance

L'architecture sépare volontairement les chemins de confiance humain et machine. Les opérateurs humains s'authentifient avec JWT et RBAC, tandis que le capteur Ubuntu transmet ses alertes à l'aide d'une clé API dédiée. Cette séparation évite de réutiliser les identifiants humains pour un composant automatisé et permet d'auditer distinctement les échecs d'authentification du capteur.

3.5 Principes de conception des interfaces

Le frontend est organisé autour d'un flux de supervision authentifié : écran de connexion dédié, tableau de bord adapté au rôle, statistiques et graphiques, table d'alertes filtrable et paginée, vue détaillée d'une alerte et export CSV. Les actions privilégiées, telles que la suppression, sont réservées à l'administrateur.

Chapitre 4 - Réalisation

4.1 Environnement de travail et technologies

Couche / domaine	Technologie
Laboratoire réseau	EVE-NG, Kali Linux, Ubuntu Server
Capture de trafic / extraction des caractéristiques	Scapy, Python
Apprentissage automatique	Random Forest scikit-learn, artefacts de modèle joblib
API backend	FastAPI, SQLAlchemy, OAuth2/JWT
Base de données	PostgreSQL
Frontend	React, Vite, Axios, Recharts, Nginx
Exploitation du service temps réel	systemd, rotating logs, JSON heartbeat
Déploiement	Docker, Docker Compose
Intégration continue	GitHub Actions

4.2 Détecteur Ubuntu en temps réel

Le détecteur temps réel s'exécute sur Ubuntu Server et observe en continu l'interface ens3. Il construit la même représentation comportementale causale que celle utilisée à l'entraînement, réalise l'inférence Random Forest puis applique des règles opérationnelles avant de transmettre les alertes.

```
nidsadmin@nids-ubuntu-server:~$ sudo systemctl status sotetel-nids --no-pager
[sudo] password for nidsadmin:
• sotetel-nids.service - SOTETEL AI-NIDS Live Behavior Detector
   Loaded: loaded (/etc/systemd/system/sotetel-nids.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-08-29 15:44:46 UTC; 2min 3s ago
     Main PID: 634 (python)
        Tasks: 5 (limit: 2211)
      Memory: 319.5M
         CPU: 22.754s
    CGroup: /system.slice/sotetel-nids.service
            └─634 /home/nidsadmin/miniforge3/envs/sotetel-live/bin/python /home/nidsadmin/sotetel-live/capture/scripts/live_be..

Aug 29 15:46:45 nids-ubuntu-server python[634]: [NORMAL] NORMAL
Aug 29 15:46:45 nids-ubuntu-server python[634]: Confidence:      97.33%
Aug 29 15:46:45 nids-ubuntu-server python[634]: Source:         0.0.0.0
Aug 29 15:46:45 nids-ubuntu-server python[634]: Destination:    255.255.255.255
Aug 29 15:46:45 nids-ubuntu-server python[634]: Protocol:        UDP
Aug 29 15:46:45 nids-ubuntu-server python[634]: Packets/sample:     1
Aug 29 15:46:45 nids-ubuntu-server python[634]: Packets/sec:      0.2
Aug 29 15:46:45 nids-ubuntu-server python[634]: 30s packets:      5
Aug 29 15:46:45 nids-ubuntu-server python[634]: 30s conn rate:    0.0
Aug 29 15:46:45 nids-ubuntu-server python[634]: 30s dst ports:    1
nidsadmin@nids-ubuntu-server:~$
```

Figure 5 - Détecteur Ubuntu en temps réel géré par systemd et exécuté comme service `sotetel-nids.service`.

Le service est activé et actif, ce qui montre que la détection en temps réel est exploitée comme un service Linux géré plutôt que comme un simple processus lancé manuellement.

```
nidsadmin@nids-ubuntu-server:~$ cat ~/sotetel-live/runtime/detector_health.json
{
  "status": "online",
  "pid": 634,
  "sensor_interface": "ens3",
  "sensor_interface_status": "up",
  "management_interface": "ens4",
  "management_interface_status": "down",
  "model_loaded": true,
  "api_reachable": false,
  "api_health_url": "http://192.168.182.1:8000/health",
  "started_at": "2026-08-29T15:45:21.471001+00:00",
  "last_heartbeat": "2026-08-29T15:48:31.899997+00:00",
  "last_activity": "2026-08-29T15:47:50.310404+00:00",
  "last_prediction": "normal",
  "last_confidence": 0.9733333333333334,
  "predictions_processed": 13
}nidsadmin@nids-ubuntu-server:~$
```

Figure 6 - Fichier JSON d'état de santé produit par le détecteur en temps réel.

Au moment de la capture, le détecteur était en ligne, ens3 était opérationnelle et le modèle était chargé. Le fichier d'état signalait également correctement ens4 indisponible et l'API non joignable, démontrant la capacité d'auto-observation pendant une panne transitoire du chemin de gestion.

Élément d'exécution	Implémentation
Chemin de déploiement	~/sotetel-live
Service systemd	sotetel-nids
Interface surveillée	ens3
Interface de gestion	ens4
Seuil normal de confiance	0.70
Warm-up	10 seconds
Cooldown	30 secondes
État de santé	runtime/detector_health.json
Journal rotatif	logs/live_detector.log
Authentification API	LIVE_API_KEY -> X-NIDS-API-Key

4.2.1 Logique d'émission des alertes

1. Capturer les paquets et former l'échantillon de la fenêtre courte courante.
2. Mettre à jour le contexte causal de la source sur les 30 secondes précédentes.
3. Prédire une classe et un niveau de confiance avec le Random Forest comportemental.
4. Ignorer les prédictions correspondant au trafic normal.
5. Ignorer les prédictions d'attaque dont le niveau de confiance est inférieur au seuil configuré.
6. Supprimer les alertes dupliquées pendant la fenêtre de cooldown.
7. Envoyer les alertes retenues vers /alerts/live avec X-NIDS-API-Key.
8. FastAPI valide l'identité du capteur et persiste l'alerte.

```

destination_ports=1
2026-08-29 14:18:15,301 | INFO | Alert suppressed | reason=warmup | class=syn_flood | src=192.168.1.30 | dst=192.168.1.20
2026-08-29 15:45:09,223 | INFO | Live detector logging initialized. Sensor API key configured.
2026-08-29 15:45:21,471 | INFO | Detector initialized | interface=ens3 | warmup=10s | threshold=0.70 | cooldown=30s
2026-08-29 15:45:21,476 | INFO | Detector started | interface=ens3 | model=/home/nidsadmin/sotetel-live/ml-model/models/random_forest_behavior_nids.pkl | api=http://192.168.182.1:8000/alerts/live | warmup=10s | threshold=0.70 | cooldown=30s
2026-08-29 15:45:21,478 | INFO | Heartbeat started | interval=5.0s | file=/home/nidsadmin/sotetel-live/runtime/detector_health.json
2026-08-29 15:45:36,039 | INFO | Prediction | class=normal | confidence=0.9333 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=1 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:45:50,229 | INFO | Prediction | class=normal | confidence=0.9267 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=2 | packets_per_second=0.9511 | source_30s_packets=3 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:45:55,436 | INFO | Prediction | class=normal | confidence=0.9333 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=4 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:46:05,801 | INFO | Prediction | class=normal | confidence=0.9733 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=5 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:46:21,318 | INFO | Prediction | class=normal | confidence=0.9733 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=5 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:46:35,654 | INFO | Prediction | class=normal | confidence=0.9267 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=2 | packets_per_second=0.9068 | source_30s_packets=3 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:46:41,053 | INFO | Prediction | class=normal | confidence=0.9333 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=4 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:46:45,278 | INFO | Prediction | class=normal | confidence=0.9733 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=5 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:47:05,523 | INFO | Prediction | class=normal | confidence=0.9733 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=5 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:47:20,714 | INFO | Prediction | class=normal | confidence=0.9267 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=2 | packets_per_second=0.8944 | source_30s_packets=3 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:47:25,892 | INFO | Prediction | class=normal | confidence=0.9333 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=4 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:47:36,101 | INFO | Prediction | class=normal | confidence=0.9733 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=5 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
2026-08-29 15:47:50,310 | INFO | Prediction | class=normal | confidence=0.9733 | src=0.0.0.0 | dst=255.255.255.255 | protocol=UDP | packet_count=1 | packets_per_second=0.2000 | source_30s_packets=5 | source_30s_connection_rate=0.0000 | source_30s_destination_ports=1
nidsadmin@nids-ubuntu-server:~$ _

```

Figure 7 - Journal rotatif structuré du détecteur montrant l'initialisation, les règles opérationnelles, les prédictions, le seuil, le warm-up et le cooldown.

Le journal enregistre le chemin de décision opérationnel. Il montre notamment une prédiction SYN flood supprimée pendant le warm-up, puis des prédictions normales après l'initialisation du détecteur avec le seuil standard de confiance fixé à 0,70.

4.2.2 Observation sur la confiance en conditions réelles

Un scénario SYN flood borné exécuté ultérieurement restait correctement classé comme `syn_flood`, mais produisait des niveaux de confiance d'environ 0,40 à 0,56, donc inférieurs au seuil opérationnel normal de 0,70. Pour un seul test d'acceptation de l'intégration, le seuil a été temporairement abaissé à 0,40, ce qui a permis à une alerte réellement générée par le détecteur d'atteindre PostgreSQL avec une confiance d'environ 0,4067. Le seuil a ensuite été rétabli à 0,70.

Interprétation d'ingénierie

Il s'agit d'un problème d'étalonnage du modèle / distribution du trafic en direct, et non d'un échec de l'authentification capteur, de l'API, de la base ou du frontend. Une classe prédite peut être correcte tout en conservant une confiance basée sur les votes du Random Forest inférieure au seuil opérationnel.

4.3 Backend FastAPI et opérations API

FastAPI constitue la frontière centrale de l'application et de la sécurité. Il charge le modèle de base au niveau paquet, expose les routes de prédiction et de gestion des alertes, persiste les enregistrements avec SQLAlchemy, applique les autorisations utilisateur/capteur et publie l'état de santé du système.

Capacité	Implémentation
Santé	/health indique l'état de préparation de l'API, de la base et du modèle.
Prédiction	POST /predict utilise le modèle paquet de base avec une autorisation Analyste ou supérieure.
Ingestion temps réel	POST /alerts/live n'accepte que les soumissions de capteurs authentifiés.
Opérations sur les alertes	Liste, détail, filtrage, pagination, statistiques et suppression administrateur.
Authentification	Connexion OAuth2 par formulaire mot de passe + jetons JWT Bearer.
Audit	GET /audit-logs réservé à l'administrateur, adossé aux enregistrements d'audit PostgreSQL.
Configuration	DATABASE_URL, MODEL_PATH, CORS, JWT et les paramètres capteur sont pilotés par l'environnement.

4.3.1 Principales routes API

Méthode	Chemin	Accès	Objectif
GET	/	Public	Réponse racine de l'API
GET	/health	Public	Santé API/base/modèle
POST	/auth/login	Public	Authentifier et émettre un JWT
GET	/auth/me	Lecteur+	Retourner l'utilisateur authentifié courant
POST	/predict	Analyste+	Prédiction du modèle paquet
POST	/alerts/live	Clé capteur	Persister une alerte du détecteur temps réel
GET	/alerts	Lecteur+	Lister les alertes
GET	/alerts/filter	Lecteur+	Filtrage et pagination côté serveur
GET	/alerts/{id}	Lecteur+	Détail d'alerte
DELETE	/alerts/{id}	Administrateur	Supprimer l'alerte
GET	/stats	Lecteur+	Statistiques du tableau de bord
GET	/audit-logs	Administrateur	Lire la piste d'audit de sécurité

4.4 Tableau de bord de supervision React

Le frontend est développé avec React, Vite, Axios et Recharts. Les builds Docker de production compilent l'application Vite et servent le contenu statique via Nginx.

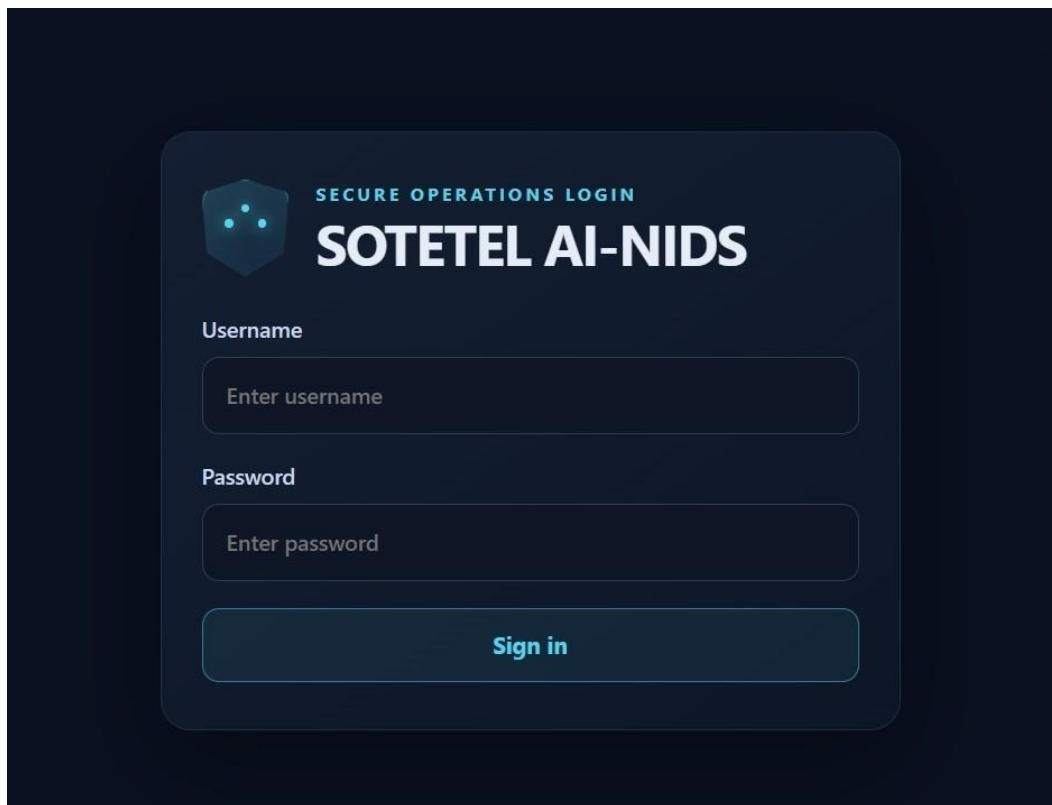


Figure 8 - Interface sécurisée de connexion aux opérations SOTETEL AI-NIDS.

Les opérateurs humains doivent s'authentifier avant d'accéder au tableau de bord de supervision. Les valeurs de mot de passe sont volontairement absentes des éléments de preuve.

- Écran de connexion sécurisé et restauration de la session JWT via sessionStorage pour le MVP.
- Interface adaptée au rôle pour les utilisateurs lecteur, analyste et administrateur.
- Six cartes statistiques sur le tableau de bord, graphiques par attaque/sévérité/protocole et table des alertes.
- Filtrage côté serveur, pagination, fenêtre de détail d'alerte et export CSV.
- Actualisation automatique toutes les cinq secondes, indication de la dernière mise à jour, affichage de l'état du backend et reprise propre après une indisponibilité temporaire de l'API.
- Bouton de suppression réservé à l'administrateur ; les rôles non administrateurs ne reçoivent pas cette action privilégiée dans l'interface.
- L'interface de connexion distingue les identifiants invalides (401) de la limitation de débit (429).
- VITE_API_BASE_URL permet de configurer l'API au moment du déploiement, avec un repli de développement local vers <http://127.0.0.1:8000>.

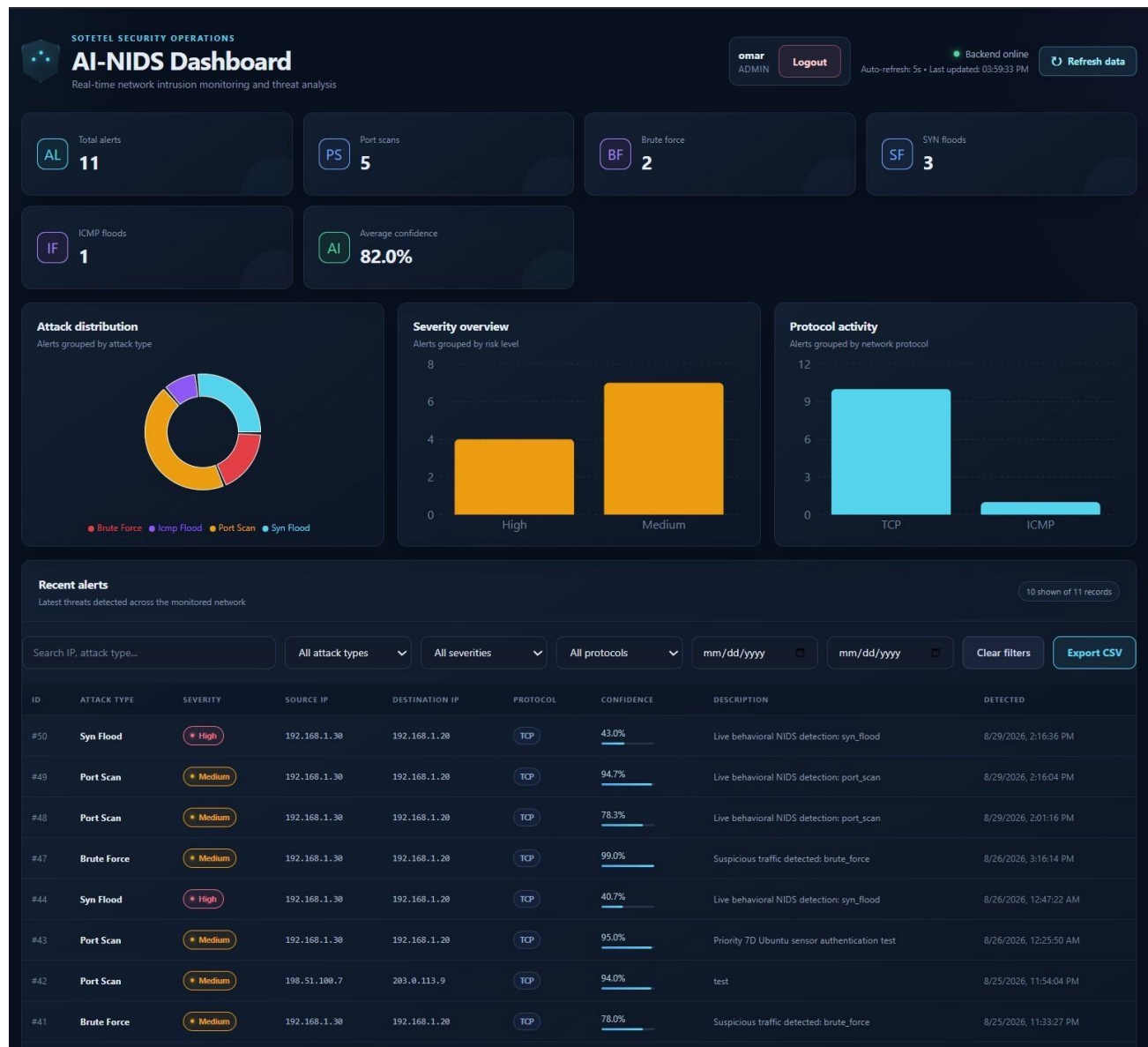


Figure 9 - Tableau de bord administrateur avec statistiques en temps réel, graphiques, état de santé, filtres et alertes récentes.

Au moment de la capture, le tableau de bord affichait 11 alertes : 5 scans de ports, 2 alertes de force brute, 3 SYN floods et 1 ICMP flood, avec une confiance moyenne de 82,0 %. L'indicateur de santé du backend était en ligne.

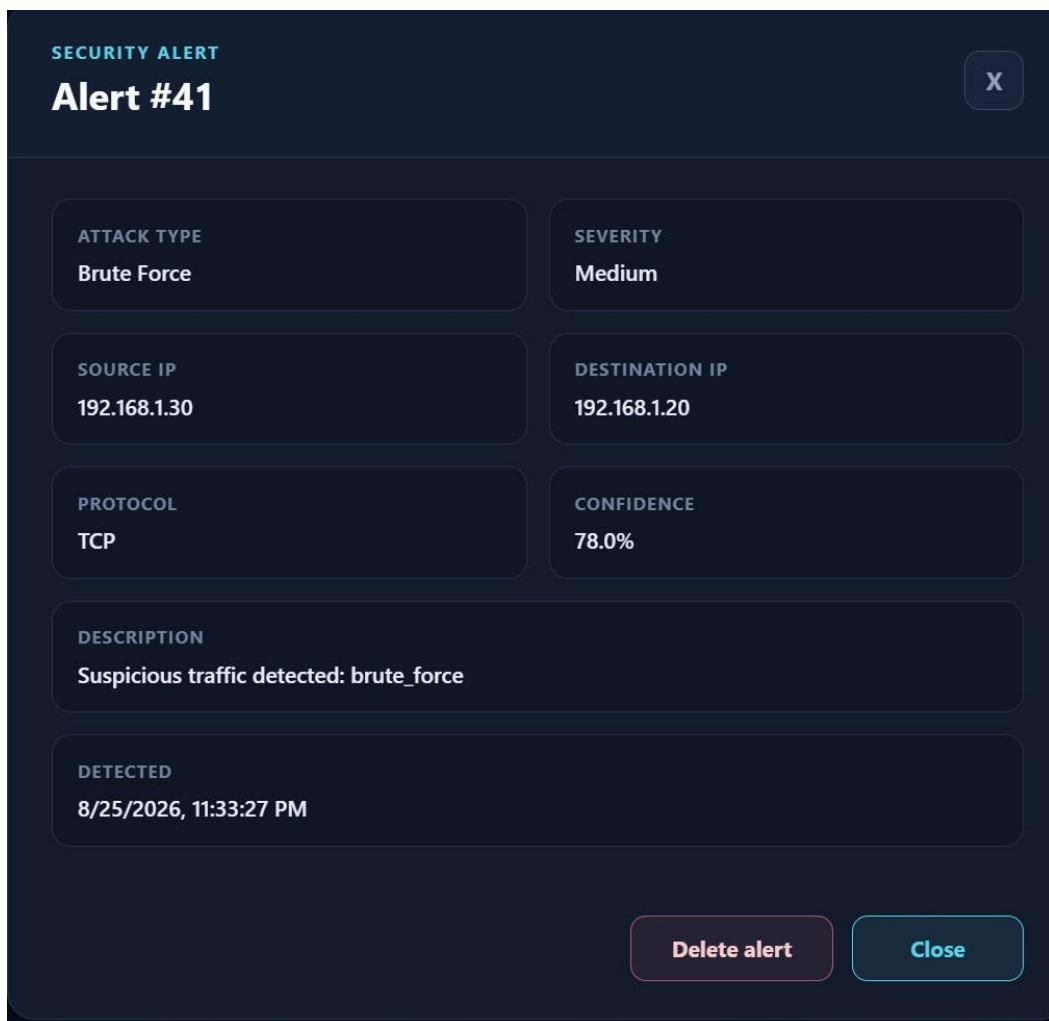


Figure 10 - Vue détaillée d'une alerte de force brute enregistrée.

Le panneau de détail présente le type d'attaque, la sévérité, les adresses IP source et destination, le protocole, la confiance, la description, l'horodatage et l'action de suppression réservée à l'administrateur.

4.5 Persistance PostgreSQL et hygiène des données

PostgreSQL stocke les alertes, les utilisateurs et les événements d'audit. Le projet contient des scripts reproductibles pour créer des utilisateurs et réinitialiser ou ensemençer les données d'alerte de démonstration, ce qui permet de préparer des démonstrations propres sans modifier manuellement des lignes SQL.

```
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform> python -c "from backend.app.database import SessionLocal; from backend.app.models import AlertModel; db=SessionLocal(); rows=db.query(AlertModel).filter(AlertModel.description.ilike('%Live behavioral NIDS detection%')).order_by(AlertModel.id.desc()).limit(10).all(); [print(f'ID={r.id} | type={r.attack_type} | severity={r.severity} | src={r.src_ip} | dst={r.dst_ip} | protocol={r.protocol} | confidence={r.confidence:.4f} | created_at={r.created_at}') for r in rows]; db.close()"
ID=50 | type=syn_flood | severity=High | src=192.168.1.30 | dst=192.168.1.20 | protocol=TCP | confidence=0.4300 | created_at=2026-08-29 14:16:36.514589
ID=49 | type=port_scan | severity=Medium | src=192.168.1.30 | dst=192.168.1.20 | protocol=TCP | confidence=0.9467 | created_at=2026-08-29 14:16:04.105697
ID=48 | type=port_scan | severity=Medium | src=192.168.1.30 | dst=192.168.1.20 | protocol=TCP | confidence=0.7833 | created_at=2026-08-29 14:01:16.698476
ID=44 | type=port_scan | severity=Medium | src=192.168.1.30 | dst=192.168.1.20 | protocol=TCP | confidence=0.4067 | created_at=2026-08-26 00:47:22.148716
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform>
```

Figure 11 - Alertes comportementales en temps réel persistées dans PostgreSQL et interrogées via le modèle applicatif.

Les preuves contiennent des événements réellement générés par le détecteur depuis 192.168.1.30 vers 192.168.1.20, notamment des alertes de scan de ports avec des confiances de 0,9467 et 0,7833 ainsi que des alertes SYN flood à 0,4300 et 0,4067.

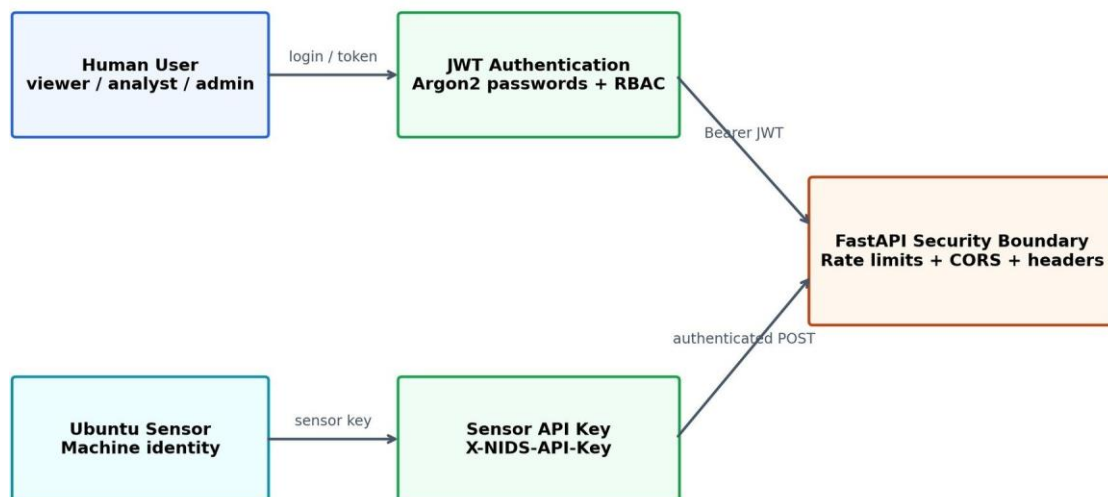
Zone de données	Objectif
alerts	Événements d'attaque détectés, sévérité, attributs réseau, confiance, description et horodatage.
users	Nom d'utilisateur, hash Argon2 du mot de passe, rôle et statut actif.
audit_logs	Échecs/succès d'authentification, refus d'autorisation, échecs capteur et événements administratifs.
Demo utilities	cleanup_demo_alerts.py et reset_demo_db.py permettent une préparation reproductible des démonstrations.
Administrateur utility	create_admin_user.py crée interactivement des utilisateurs lecteur/analyste/administrateur sans mot de passe codé en dur.

Persistence rule

Docker Compose utilise un volume PostgreSQL nommé. Les redémarrages normaux des conteneurs conservent les données. La suppression du volume persistant supprime la base conteneurisée et nécessite donc une planification explicite de sauvegarde/nettoyage.

4.6 Mise en œuvre de la sécurité applicative

Separate Human and Machine Authentication Paths



Audit events: LOGIN_SUCCESS / LOGIN_FAILURE / AUTHORIZATION_DENIED / ALERT_DELETED / SENSOR_AUTH_FAILURE

Figure 12 - Séparation volontaire entre le chemin JWT/RBAC humain et le chemin de clé capteur machine.

4.6.1 Authentification et autorisation

Contrôle	État final
Stockage des mots de passe	Hachage Argon2 via pwdlib.
JWT	Jetons d'accès Bearer avec durée configurée ; les jetons expirés ou invalides sont rejetés.
RBAC	Hierarchie lecteur < analyste < administrateur appliquée par les dépendances backend.
Identité du capteur	X-NIDS-API-Key dédiée, comparaison en temps constant, séparée du JWT humain.

Contrôle	État final
Secrets	Les valeurs réelles restent dans les fichiers .env locaux et sont exclues de Git.

4.6.2 Journalisation d'audit

Les actions sensibles pour la sécurité sont persistées dans la table `audit_logs`. Les événements validés pendant le durcissement incluent `LOGIN_SUCCESS`, `LOGIN_FAILURE`, `AUTHORIZATION_DENIED`, `ALERT_DELETED` et `SENSOR_AUTH_FAILURE`. L'API d'audit est réservée aux administrateurs.

```
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform> python -c "from sqlalchemy import text; from backend.app.database import SessionLocal; db=SessionLocal(); rows=db.execute(text('SELECT id, event_type, username, role, resource, action, status, created_at FROM audit_logs ORDER BY id DESC LIMIT 15')).all(); [print(row) for row in rows]; db.close()"
```

```
(30, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 29, 14, 59, 23, 128421))
(29, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 29, 14, 58, 6, 895492))
(28, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 29, 13, 51, 6, 133883))
(27, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 29, 13, 50, 52, 924186))
(26, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 29, 13, 50, 44, 674498))
(25, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 29, 13, 50, 38, 235112))
(24, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 26, 16, 58, 13, 308833))
(23, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 26, 16, 58, 4, 654925))
(22, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 26, 16, 38, 12, 443861))
(21, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 26, 16, 33, 32, 478386))
(20, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 26, 15, 47, 35, 238657))
(19, 'LOGIN_SUCCESS', 'omar', 'admin', '/auth/login', 'login', 'success', datetime.datetime(2026, 8, 26, 15, 16, 4, 919428))
(18, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 26, 15, 15, 50, 225597))
(17, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 26, 14, 47, 10, 608544))
(16, 'LOGIN_FAILURE', 'omar', None, '/auth/login', 'login', 'failure', datetime.datetime(2026, 8, 26, 14, 47, 10, 557030))
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform>
```

Figure 13 - Preuve du journal d'audit PostgreSQL montrant des événements d'authentification réussis et échoués.

Les enregistrements visibles démontrent la persistance des événements `LOGIN_SUCCESS` et `LOGIN_FAILURE` avec les champs utilisateur, rôle, ressource, action, statut et horodatage. Le sous-système d'audit prend également en charge les événements d'autorisation, de suppression et d'authentification du capteur validés pendant le durcissement.

4.6.3 Limitation de débit

Route	Limite
/auth/login	5/minute
/predict	30/minute
/audit-logs	30/minute
/alerts/live	60/minute

4.6.4 Durcissement HTTP et navigateur

- Liste d'origines CORS explicitement autorisées pour `http://localhost:5173` et `http://127.0.0.1:5173`, configurable via `CORS_ALLOWED_ORIGINS`.
- X-Content-Type-Options: `nosniff`.
- X-Frame-Options: `DENY`.
- Referrer-Policy: `no-referrer`.
- Permissions-Policy restrictive pour les capacités caméra, microphone, géolocalisation, USB et paiement.
- Cache-Control: `no-store` sur les réponses sensibles d'authentification, d'utilisateur et d'audit.

```
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform> $health = Invoke-WebRequest `
>> -Uri "http://127.0.0.1:8000/health" `
>> -UseBasicParsing
>>
>> Write-Host "STATUS:" $health.StatusCode
>> Write-Host "BODY:" $health.Content
>> Write-Host "X-Content-Type-Options:" $health.Headers["X-Content-Type-Options"]
>> Write-Host "X-Frame-Options:" $health.Headers["X-Frame-Options"]
>> Write-Host "Referrer-Policy:" $health.Headers["Referrer-Policy"]
>> Write-Host "Permissions-Policy:" $health.Headers["Permissions-Policy"]
STATUS: 200
BODY: {"status":"healthy","api":"running","database":"connected","model":"loaded"}
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Permissions-Policy: camera=(), microphone=(), geolocation=(), usb=(), payment=()
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform>
```

Figure 14 - Vérification en exécution de l'état de santé de l'API et des en-têtes de sécurité HTTP.

Le point de terminaison de santé a renvoyé HTTP 200 avec l'API active, la base connectée et le modèle chargé. La réponse confirmait également nosniff, l'interdiction d'affichage en frame, no-referrer et une Permissions-Policy restrictive.

- HSTS n'est volontairement pas activé pour le développement HTTP local ; HTTPS/TLS reste un travail de déploiement futur.

Chapitre 5 - Intelligence artificielle et détection d'intrusions

5.1 Capture des données et extraction des caractéristiques

Les captures de paquets ont été collectées lors de sessions EVE-NG contrôlées puis traitées avec Scapy. Le projet a d'abord extrait des champs au niveau paquet, avant d'introduire des caractéristiques comportementales causales adaptées à l'inférence en temps réel.

5.1.1 Modèle de base au niveau paquet

- Les champs comprennent l'horodatage, les IP source/destination, le protocole, les ports source/destination et la taille du paquet.
- Le modèle de base utilise le protocole encodé, src_port, dst_port et packet_size.
- Le modèle paquet reste disponible pour l'API POST /predict et pour comparaison avec l'approche comportementale temps réel.

5.1.2 Extraction comportementale causale

Élément de conception	Paramètre final
Fenêtre courte d'observation	5 secondes
Fenêtre glissante de contexte source	30 secondes
Nombre maximal de paquets par échantillon	50
Politique d'historique	Causal uniquement - aucune fuite d'information future
Initiateur TCP	SYN activé et ACK non activé
Initiateur UDP	Premier émetteur
Initiateur ICMP	Émetteur de requête echo

Le modèle comportemental utilise 28 caractéristiques. Les familles les plus importantes comprennent le nombre et le débit des connexions, le comportement SYN et RST, la diversité des ports de destination, les débits de paquets et d'octets, les statistiques de taille des paquets et le contexte source sur 30 secondes. Les identifiants de session, horodatages, adresses IP et étiquettes ne sont pas utilisés comme caractéristiques du modèle.

5.2 Modèle de base au niveau paquet

Le Random Forest de base est conservé comme référence compacte et alimente le point de terminaison de prédiction de l'API. Le projet avait initialement établi un modèle de base à trois classes au niveau paquet ; l'artefact final d'évaluation fournit la preuve de référence pour ce rapport : 735 échantillons de test contrôlés, soit 245 par classe, avec une exactitude de 99,05 %.

```

(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform> Write-Host "=== BASE PACKET MODEL - CLASSIFICATION REPORT ==="
>> Get-Content ml-model\evaluation\classification_report.csv
>>
>> Write-Host "`n=== CONFUSION MATRIX ==="
>> Get-Content ml-model\evaluation\confusion_matrix.csv
=== BASE PACKET MODEL - CLASSIFICATION REPORT ===
,precision,recall,f1-score,support
brute_force,0.976,0.9959183673469387,0.9858585858585859,245.0
normal,0.9958333333333333,0.9755102040816327,0.9855670103092784,245.0
port_scan,1.0,1.0,1.0,245.0
accuracy,0.9904761904761905,0.9904761904761905,0.9904761904761905,0.9904761904761905
macro_avg,0.9906111111111112,0.9904761904761905,0.9904751987226215,735.0
weighted_avg,0.9906111111111111,0.9904761904761905,0.9904751987226216,735.0

=== CONFUSION MATRIX ===
,brute_force,normal,port_scan
brute_force,244,1,0
normal,6,239,0
port_scan,0,0,245
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform>

```

Figure 15 - Rapport de classification et matrice de confusion du modèle paquet de base sauvegardé.

L'artefact final affiche une exactitude de 99,05 % sur 735 échantillons de test contrôlés. Il s'agit d'une preuve d'évaluation en laboratoire et non d'une garantie d'exactitude en production.

Métrique / classe	Résultat
Preuve d'évaluation	Rapport de classification sauvegardé actuel
Support de test	735 échantillons (245 par classe)
Exactitude globale	99.05%
brute_force	Précision 97.60% / Rappel 99.59% / F1 98.59%
normal	Précision 99.58% / Rappel 97.55% / F1 98.56%
port_scan	Précision 100.00% / Rappel 100.00% / F1 100.00%
Erreurs de la matrice de confusion	7 / 735 au total : 1 brute_force -> normal ; 6 normal -> brute_force ; port_scan parfait.

Interprétation

Le modèle de base montre de très bonnes performances sur le jeu contrôlé au niveau paquet, mais il ne fournit pas le même contexte temporel que le modèle comportemental en direct. Il est donc conservé comme référence/modèle API plutôt que comme modèle principal du NIDS temps réel.

5.3 Modèle comportemental à cinq classes et validation

Le modèle comportemental est le modèle principal du détecteur en temps réel. Il classe cinq étiquettes : brute_force, icmp_flood, normal, port_scan et syn_flood. Le jeu final contient 775 échantillons issus de 21 sessions de capture, utilise 28 caractéristiques et applique une validation séparée par session.

```
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform> $m = Import-Csv "ml-model\evaluation\behavior_multi_session\split_metrics.csv"
>>
>> $meanAcc = ($m | ForEach-Object { [double]$_ .accuracy } | Measure-Object -Average).Average
>> $meanBal = ($m | ForEach-Object { [double]$_ .balanced_accuracy } | Measure-Object -Average).Average
>> $meanF1 = ($m | ForEach-Object { [double]$_ .macro_f1 } | Measure-Object -Average).Average ...
>> Write-Host ("Mean macro F1      : {0:P2}" -f $meanF1)
>> Write-Host ("Worst split       : {0:P2} ({1})" -f ([double]$worst.accuracy), $worst.split_id)
>> Write-Host ""
>> Write-Host ("Brute-force recall : {0:P2}" -f $bfRecall)
>> Write-Host ("Normal recall    : {0:P2}" -f $normRecall)
>> Write-Host ("Port-scan recall  : {0:P2}" -f $scanRecall)
>> Write-Host ("SYN-flood recall  : {0:P2}" -f $synRecall)
>> Write-Host ("ICMP-flood recall : {0:P2}" -f $icmpRecall)
>> Write-Host ""
>> Write-Host "=== AGGREGATE CONFUSION MATRIX ==="
>> Get-Content "ml-model\evaluation\behavior_multi_session\aggregate_confusion_matrix.csv"
=== FIVE-CLASS BEHAVIORAL MODEL ===
Behavioral samples : 775
Capture sessions   : 21
Model features     : 28
Holdout experiments: 144

Mean accuracy      : 99.48%
Balanced accuracy  : 99.53%
Mean macro F1      : 99.24%
Worst split        : 96.76% (split_60)

Brute-force recall : 100.00%
Normal recall      : 100.00%
Port-scan recall   : 97.67%
SYN-flood recall   : 100.00%
ICMP-flood recall  : 100.00%

=== AGGREGATE CONFUSION MATRIX ===
,brute_force,normal,port_scan,syn_flood,icmp_flood
brute_force,5832,0,0,0,0
normal,0,3240,0,0,0
port_scan,6,188,7546,0,0
syn_flood,0,0,0,10800,0
icmp_flood,0,0,0,0,6096
(.venv) PS C:\Users\obelh\OneDrive\Documents\GitHub\Sotetel-AI-NIDS-Platform>
```

Figure 16 - Résumé de l'évaluation multi-session du modèle comportemental à cinq classes et matrice de confusion agrégée.

Sur 144 expériences de holdout au niveau session, l'exactitude moyenne atteint 99,48 %, l'exactitude équilibrée 99,53 %, le macro F1 moyen 99,24 %, et le pire split 96,76 %. Les erreurs résiduelles se concentrent principalement sur les sessions de scan de ports.

Classe	Échantillons	Exemples de sessions sources
syn_flood	225	syn_flood_01, _02, _03
port_scan	215	port_scan_01, _02, _03, _04
icmp_flood	127	icmp_flood_01, _02, _03
normal	120	HTTP, SSH, UDP, sessions ping
brute_force	88	brute_force_01, _02, _03, _04
TOTAL	775	21 sessions de capture

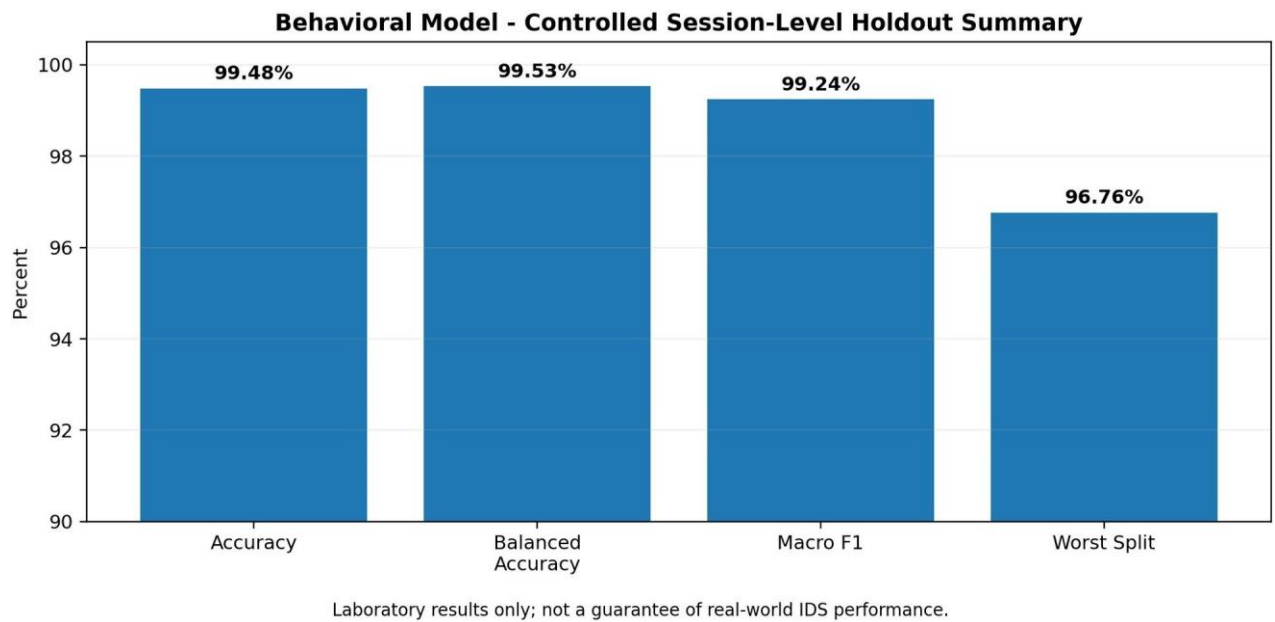


Figure 17 - Résumé final de la validation contrôlée par session.

Élément d'évaluation	Résultat
Échantillons comportementaux	775
Sessions de capture	21
Caractéristiques du modèle	28
Expériences de holdout au niveau session	144
Exactitude moyenne	99.48%
Exactitude équilibrée moyenne	99.53%
Macro F1 moyen	99.24%
Exactitude du pire split	96.76%

Les rappels moyens observés dans l'évaluation contrôlée étaient : brute_force 100 %, normal 100 %, port_scan 97,67 %, syn_flood 100 % et icmp_flood 100 %. Un test ciblé de robustesse normal/ICMP a également obtenu 100 % de classification sur des échantillons de ping normal et d'ICMP flood tenus à l'écart.

Précision importante

Les résultats au niveau session réutilisent l'ensemble fini des sessions de capture du laboratoire selon différentes combinaisons de holdout. Les métriques agrégées sont utiles pour analyser la robustesse interne, mais ne doivent pas être présentées comme un nombre d'événements réels indépendants ni comme une exactitude garantie en production.

5.4 Interprétation des résultats

Le modèle de base au niveau paquet reste utile comme point de comparaison pour l'API, mais le modèle comportemental constitue le détecteur principal en temps réel car il représente le comportement temporel du trafic grâce à une fenêtre courte causale et à un contexte source glissant. Les métriques de holdout rapportées sont élevées dans le jeu de données contrôlé du laboratoire ; toutefois, le projet distingue explicitement ces résultats de toute garantie de performance dans un environnement réel. Les observations de confiance lors du SYN flood en direct montrent également que la justesse de la classe prédite et l'étalonnage du niveau de confiance opérationnel sont deux problématiques distinctes.

Chapitre 6 - DevSecOps, déploiement et validation

6.1 Docker et déploiement reproductible

Les services applicatifs ont été conteneurisés après le durcissement de sécurité. La pile Compose contient PostgreSQL, FastAPI et le frontend React/Nginx. Le capteur Ubuntu EVE-NG en temps réel reste externe.

Service	Image / environnement	Comportement principal
db	postgres:17-alpine	Volume persistant, contrôle de santé, nom de service interne db.
backend	python:3.13-slim	Installe les dépendances, copie les fichiers backend et modèle, expose le port 8000 et fournit un contrôle de santé.
frontend	Node 22 build -> nginx:alpine	Exécute npm ci/build dans l'étape builder et sert dist via Nginx sur le port hôte 5173.

Dans Compose, FastAPI se connecte à PostgreSQL via le nom DNS Docker db plutôt que localhost. Le navigateur continue d'appeler `http://localhost:8000` puisqu'il s'exécute en dehors du réseau Docker.

6.1.1 Jalons de conteneurisation validés

- L'image backend a été construite avec succès et contenait `/app/ml-model/models/random_forest_nids.pkl`.
- Le build de production multi-étapes du frontend a été réalisé avec succès.
- Les services PostgreSQL, backend et frontend ont atteint un état sain/en exécution.
- `/health` a indiqué que l'API était active, la base connectée et le modèle chargé.
- Le frontend a renvoyé HTTP 200 via Nginx.
- Un utilisateur administrateur Docker et quatre alertes de démonstration ont été créés dans la nouvelle base conteneurisée.
- Après `docker compose restart`, les quatre mêmes alertes étaient toujours présentes, démontrant la persistance du volume nommé lors de redémarrages normaux.

6.2 Intégration continue avec GitHub Actions

Le dépôt contient `.github/workflows/ci.yml`. Le workflow s'exécute lors des push et pull requests vers main et peut aussi être lancé manuellement. La dernière exécution capturée (#7) s'est terminée avec succès en 1 min 45 s : contrôles backend 44 s, frontend 10 s et builds Docker 52 s. Le workflow valide l'intégrité Python/backend, le chemin applicatif adossé à PostgreSQL, le lint/build du frontend et les deux images Docker.

Tâche	Contrôles
Contrôles backend	Installation Python 3.13 ; dépendances ; service PostgreSQL 17 ; existence du fichier modèle ; compilation backend ; test rapide du schéma ; import FastAPI.
Contrôles frontend	Node 22 ; npm ci ; ESLint ; build Vite de production.
Contrôles des builds Docker	Build de l'image backend et de l'image frontend après réussite des contrôles backend/frontend.

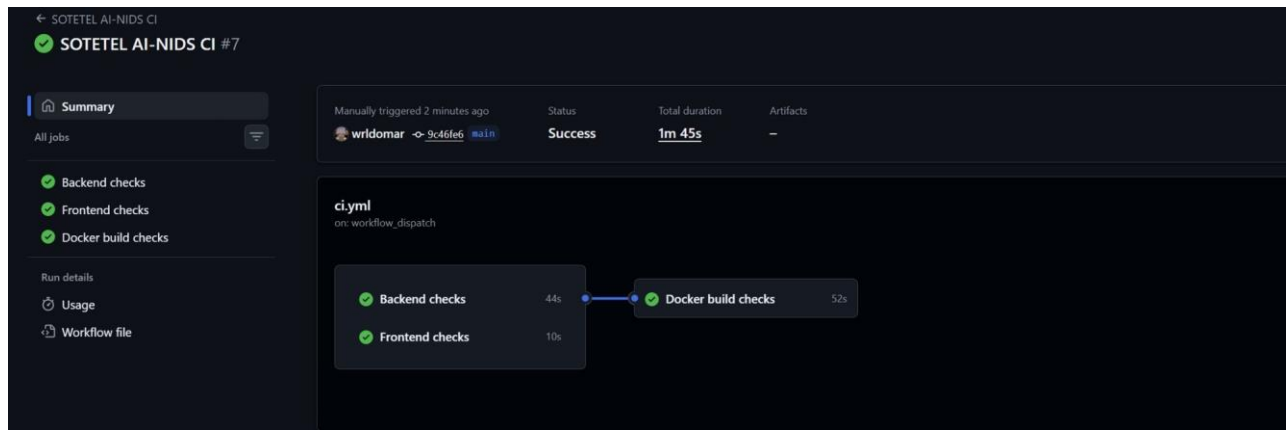


Figure 18 - Exécution finale GitHub Actions CI (#7) : contrôles backend, frontend et builds Docker tous réussis.

Périmètre de la CI

Le workflow actuel couvre l'intégration continue. Le déploiement automatique en production (CD) n'est pas implémenté et le rapport ne prétend pas le contraire.

6.3 Validation de bout en bout et preuves d'ingénierie

La validation a été effectuée de manière incrémentale afin d'isoler chaque sous-système en cas d'échec. Une fois un jalon validé, le projet a adopté une politique de test fermé : ne répéter que les contrôles affectés par des modifications ultérieures du code, de la configuration, des dépendances ou du réseau.

Domaine	Preuve de validation
ML	Modèle paquet de base et modèle comportemental à cinq classes entraînés/évalués sur des données de laboratoire contrôlées.
Capture temps réel	Ubuntu ens3 a observé du trafic EVE-NG normal et malveillant.
Pipeline capteur	Les alertes générées automatiquement par le détecteur ont atteint FastAPI/PostgreSQL/le tableau de bord lors des exécutions validées.
Fiabilité	Journal rotatifs, heartbeat, systemd startup, stop, crash restart and boot enablement validated.
Sécurité	JWT, comportement avec jeton absent/invalid, refus RBAC, échec/succès de clé capteur, événements d'audit, limitation 429, CORS et en-têtes validés.
Déploiement	Pile Docker Compose saine ; modèle chargé ; base persistante après redémarrage normal.
CI	Les jobs GitHub Actions backend, frontend et Docker ont tous réussi.

6.3.1 Preuves d'alertes réelles en temps réel

Les preuves finales montrent plusieurs alertes réellement générées par le détecteur et persistées à travers toute la chaîne. Le 29 août 2026, la base contenait une alerte port_scan à 94,67 % de confiance et une alerte SYN flood à 43,00 % depuis Kali 192.168.1.30 vers Ubuntu 192.168.1.20. L'événement SYN flood de confiance plus faible a été accepté pendant une démonstration d'intégration contrôlée avec un seuil temporairement abaissé, tandis que le seuil opérationnel normal reste 0,70. Cela confirme le chemin capture de paquets -> inférence comportementale -> POST capteur authentifié -> FastAPI -> PostgreSQL -> tableau de bord React, tout en documentant la limite d'étalonnage du modèle sur le trafic en direct.

6.4 Principales leçons de débogage et de récupération

6.4.1 Bug de soumission des alertes du détecteur

Pendant le durcissement, le détecteur classait correctement le trafic SYN flood, mais une instruction `return` mal indentée faisait quitter `send_live_alert()` avant l'envoi POST vers l'API. Un POST manuel a confirmé que le réseau Ubuntu, FastAPI et PostgreSQL fonctionnaient correctement, isolant le défaut dans le flux de contrôle du détecteur. La correction de l'ordre des gardes a restauré la persistance automatique.

6.4.2 Saturation du disque Ubuntu

Pendant la validation de l'authentification du capteur, le système de fichiers racine Ubuntu a atteint 100 % d'utilisation et provoqué une erreur d'écriture `curl`. Les caches de paquets, journaux, Conda et Snap ont été nettoyés, libérant de l'espace. L'utilisation disque reste un point opérationnel à surveiller et, pour une utilisation prolongée, le système de fichiers de la VM devrait être agrandi.

6.4.3 Récupération des interfaces réseau

Panne	Récupération
Ubuntu ens4 perd son adresse de gestion	Remonter ens4, renouveler DHCP, vérifier 192.168.182.x/24, pinguer 192.168.182.1 puis appeler /health.
Kali eth0 perd son adresse de laboratoire	Remonter eth0, restaurer 192.168.1.30/24 puis pinguer Ubuntu 192.168.1.20.
JWT expiré	Se réauthentifier ; ne pas traiter l'expiration du jeton comme un défaut d'identifiants.
HTTP 429 lors de la connexion	Attendre la fin de la fenêtre de limitation plutôt que recréer les comptes.
Moteur Docker indisponible	Vérifier WSL2 et Docker Desktop ; docker version doit afficher les sections Client et Server.

6.4.4 Conflit entre VMware EVE-NG et Docker/WSL2 sur l'hôte

Sur le poste Windows, EVE-NG dans VMware nécessite la virtualisation imbriquée Intel VT-x/EPT, tandis que Docker Desktop dépend de WSL2 et de l'hyperviseur Microsoft. Après activation de WSL2, VMware ne pouvait plus reprendre un ancien état CPU suspendu ni exposer VT-x/EPT imbriqué lorsque l'hyperviseur Microsoft était actif. Le flux de travail pratique consiste à valider Docker et EVE-NG dans des modes de démarrage hôte séparés, ou à utiliser un second hôte/serveur distant lorsqu'une exécution simultanée est nécessaire.

6.5 Limites actuelles

- Prototype de laboratoire et non IDS de production.
- Jeu de données personnalisé avec une diversité réelle limitée et un nombre fini de sessions de capture indépendantes.
- L'étalonnage de la confiance en direct et la généralisation entre environnements doivent être améliorés.
- La capacité disque de la VM Ubuntu doit être surveillée.
- HTTPS/TLS n'est pas déployé pour une exposition externe en production.
- Le frontend MVP conserve le JWT dans `sessionStorage`.
- SQLAlchemy `create_all` est utilisé à la place d'un framework formel de migration tel qu'Alembic.
- Les métriques centralisées, l'observabilité et l'agrégation de journaux à long terme peuvent être développées davantage.

- La CI valide les builds et les imports, mais une suite automatisée backend/API plus complète reste souhaitable.
- Docker Desktop et EVE-NG virtualisé de manière imbriquée entrent en conflit sur l'hôte Windows actuel lorsque les deux nécessitent simultanément la virtualisation matérielle.

6.6 Perspectives

Priorité	Travail à réaliser	Bénéfice attendu
Élevée	Collecter davantage de sessions indépendantes avec différents débits, profils temporels et conditions réseau.	Réduire l'écart de distribution en direct et améliorer la généralisation.
Élevée	Étalonner la confiance du Random Forest et choisir les seuils à partir d'une analyse de validation de type ROC.	Réduire les faux négatifs dus à des seuils conservateurs tout en contrôlant les faux positifs.
Élevée	Ajouter HTTPS/TLS et une gestion renforcée des secrets pour un déploiement hors laboratoire.	Protéger les identifiants et le trafic en dehors d'un environnement local de confiance.
Moyenne	Introduire les migrations de base de données Alembic.	Rendre l'évolution du schéma et le déploiement plus sûrs et reproductibles.
Moyenne	Étendre les tests automatisés API/sécurité et l'analyse des dépendances/vulnérabilités.	Augmenter la couverture de régression et la visibilité sur la chaîne d'approvisionnement.
Moyenne	Ajouter des métriques centralisées, une observabilité structurée et des politiques de rétention.	Améliorer le diagnostic opérationnel.
Optionnel	Mises à jour temps réel du tableau de bord par WebSocket/SSE.	Réduire la latence liée au polling.
Optionnel	EVE-NG distant ou hôte Linux de déploiement séparé / évaluation Kubernetes.	Éviter les conflits de virtualisation du poste et permettre des expérimentations à plus grande échelle.

Conclusion générale

Le projet SOTETEL AI-NIDS s'inscrit dans une démarche de cybersécurité visant à renforcer la supervision réseau par l'intégration de techniques d'intelligence artificielle. Il repose sur le principe d'un système de détection d'intrusions réseau capable d'observer le trafic, d'en extraire des caractéristiques comportementales et d'identifier automatiquement des activités susceptibles de correspondre à des intrusions. La plateforme a ainsi pour finalité de transformer des flux réseau complexes en alertes compréhensibles et exploitables par les opérateurs de sécurité.

L'architecture de SOTETEL AI-NIDS associe un environnement réseau contrôlé EVE-NG, un capteur Ubuntu en temps réel, un modèle Random Forest multiclass, un backend FastAPI, une base de données PostgreSQL et une interface React. Ces composants forment une chaîne cohérente dans laquelle le trafic est observé, analysé, classifié, filtré puis transmis sous forme d'alertes authentifiées vers la couche applicative. La détection couvre le trafic normal ainsi que quatre catégories d'attaques : le scan de ports, la force brute, le SYN flood et l'ICMP flood.

Au-delà du modèle d'intelligence artificielle, la plateforme adopte une approche globale de la sécurité. La séparation entre l'authentification des utilisateurs et celle du capteur, le contrôle d'accès par rôles, la journalisation d'audit et la persistance des événements contribuent à assurer la traçabilité des actions et la protection de la chaîne de supervision. Le tableau de bord centralise les alertes, les statistiques et les informations nécessaires à leur analyse afin de fournir une vision synthétique de l'activité du réseau.

SOTETEL AI-NIDS constitue ainsi une plateforme intégrée combinant réseaux, intelligence artificielle, cybersécurité, développement web, bases de données et déploiement conteneurisé. Son intérêt réside dans cette combinaison : l'intelligence artificielle n'est pas utilisée comme un module isolé, mais comme un élément d'une architecture complète de surveillance. Le projet illustre ainsi une approche moderne de la détection d'intrusions, orientée vers l'automatisation de l'analyse, la centralisation des alertes et l'aide à la supervision des infrastructures réseau dans un environnement de laboratoire contrôlé.

Annexes

Annexe A - Déroulement recommandé de la démonstration

Une bonne démonstration doit prouver chaque sous-système majeur une seule fois plutôt que de rejouer l'ensemble des tests historiques.

9. Afficher le dépôt GitHub et l'exécution GitHub Actions CI réussie.
10. Afficher les services Docker Compose en état sain (ou, en mode EVE-NG, préciser que Docker a été validé indépendamment).
11. Afficher FastAPI /health renvoyant API active, base connectée et modèle chargé.
12. Ouvrir la page de connexion et s'authentifier avec un compte administrateur.
13. Présenter les cartes, graphiques, table des alertes, filtres, pagination et détail d'alerte du tableau de bord.
14. Expliquer le RBAC lecteur / analyste / administrateur et montrer que la suppression est réservée à l'administrateur.
15. Afficher les journaux d'audit et expliquer les événements de sécurité persistés.
16. Afficher la topologie EVE-NG et les rôles des interfaces Ubuntu ens3/ens4.
17. Afficher `sudo systemctl status sotetel-nids` et le JSON d'état de santé.
18. Générer du trafic normal et montrer qu'il est classé normal sans persistance d'alerte d'attaque.
19. Exécuter une attaque bornée (scan de ports ou SYN flood) et observer les journaux du détecteur en temps réel.
20. Si la confiance en direct est inférieure à 0,70, expliquer la suppression par seuil plutôt que d'affirmer que le frontend a échoué. N'utiliser un seuil temporaire de 0,40 que pour une démonstration d'intégration volontaire, puis rétablir 0,70.
21. Terminer en montrant l'alerte persistée et expliquer le chemin de bout en bout depuis le trafic jusqu'au tableau de bord.

A.1 Explication du seuil de confiance prête pour la soutenance

Explication suggérée

Le Random Forest choisit d'abord la classe la plus probable. Le détecteur temps réel applique ensuite un seuil opérationnel de confiance séparé. Une prédiction peut donc être correctement classée comme attaque tout en étant supprimée si sa confiance est inférieure à 0,70. L'écart observé en direct relève de l'étalonnage/généralisation et est documenté comme travail ML futur.

Annexe B - Référence des commandes opérationnelles

B.1 Mode de démonstration EVE-NG natif

Système	Commande / vérification
Backend Windows	<code>python -m uvicorn backend.app.main:app --host 0.0.0.0 --port 8000</code>
Frontend Windows	<code>cd frontend ; npm run dev</code>
État du détecteur Ubuntu	<code>sudo systemctl statut sotetel-nids --no-pager</code>
Journaux du détecteur Ubuntu	<code>tail -f ~/sotetel-live/logs/live_detector.log</code>
État de santé Ubuntu	<code>cat ~/sotetel-live/runtime/detector_health.json</code>
Vérification API Ubuntu	<code>curl --max-time 5 http://192.168.182.1:8000/health</code>
Connectivité Kali	<code>ping -c 3 192.168.1.20</code>

B.2 Récupération des interfaces

Cas	Commandes
Ubuntu ens4	<pre> sudo ip link set ens4 up sudo dhclient -v ens4 ip -br addr ping -c 3 192.168.182.1 curl --max-time 5 http://192.168.182.1:8000/health </pre>
Kali eth0	<pre> sudo ip link set eth0 up sudo ip addr add 192.168.1.30/24 dev eth0 ip -br addr ping -c 3 192.168.1.20 </pre>

B.3 Exemples de trafic de laboratoire borné et autorisé

Scénario	Commande	Explication
Scan de ports	<code>sudo nmap -sS -Pn -p 1-1000 192.168.1.20</code>	Scan SYN borné sur les ports 1 à 1000.
SYN flood	<code>sudo nping --tcp -p 8080 --flags syn --rate 500 -c 3000 192.168.1.20</code>	500 paquets/s, arrêt après 3000 paquets.
ICMP flood	<code>sudo nping --icmp --rate 200 -c 2000 192.168.1.20</code>	200 paquets/s, arrêt après 2000 paquets.
Ping normal	<code>ping -c 5 192.168.1.20</code>	Cinq requêtes ICMP bénignes pour établir le comportement de référence.

Annexe C - Noms des variables d'environnement

Domaine	Variables
Backend	DATABASE_URL, MODEL_PATH, JWT_SECRET_KEY, JWT_ALGORITHM, JWT_ACCESS_TOKEN_MINUTES, NIDS_SENSOR_API_KEY, CORS_ALLOWED_ORIGINS
Base Docker	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD
Détecteur temps réel	LIVE_API_ALERT_URL, LIVE_INTERFACE, LIVE_ALERT_CONFIDENCE_THRESHOLD, LIVE_WARMUP_SECONDS, LIVE_ALERT_COOLDOWN, LIVE_API_TIMEOUT_SECONDS, LIVE_LOG_LEVEL, LIVE_LOG_MAX_BYTES, LIVE_LOG_BACKUP_COUNT, LIVE_API_HEALTH_URL, LIVE_HEARTBEAT_INTERVAL_SECONDS, LIVE_API_KEY

Gestion des secrets

Les noms de variables peuvent être documentés ; les mots de passe réels, secrets JWT, clés capteur et identifiants de base de production ne doivent jamais apparaître dans le dépôt, les captures du rapport ou la présentation.

Annexe D - Matrice d'acceptation de sécurité

Contrôle	Comportement attendu	Validation
Connexion valide	JWT émis	RÉUSSI
Connexion invalide	401 + événement d'échec d'audit	RÉUSSI
Jeton expiré/invalide	Validation des identifiants rejetée	RÉUSSI

Contrôle	Comportement attendu	Validation
Action lecteur -> administrateur	403 + audit AUTHORIZATION_DENIED	RÉUSSI
Clé capteur absente	401 + SENSOR_AUTH_FAILURE	RÉUSSI
Clé capteur incorrecte	403 + SENSOR_AUTH_FAILURE	RÉUSSI
Soumission capteur correcte	Alerte temps réel persistée	RÉUSSI
Limitation de connexion	La 6e tentative invalide rapide a renvoyé 429	RÉUSSI
Origine CORS de confiance	Access-Control-Allow-Origin renvoyé	RÉUSSI
Origine non fiable	Aucun en-tête d'autorisation CORS	RÉUSSI
En-têtes de sécurité	nosniff / DENY / no-referrer / permissions restrictives	RÉUSSI

Annexe E - Registre final des preuves

Le paquet final de preuves utilise des noms de fichiers ordonnés afin de réutiliser les mêmes artefacts de manière cohérente dans le README, le rapport technique et la présentation de soutenance. Treize captures fournies sont intégrées directement dans ce rapport ; les captures du tableau de bord Viewer/RBAC et de Docker Compose ont été collectées pendant le workflow de preuves mais ne faisaient pas partie du lot d'images téléversé pour cette version du rapport.

#	Preuve	Ce que cela prouve	Statut dans ce rapport
01	Topologie EVE-NG	Laboratoire autorisé et chemin réseau	Intégré - Fig. 5
02	Détecteur Ubuntu systemd	Service géré de détection temps réel	Intégré - Fig. 8
03	État de santé du détecteur	Observabilité en exécution et état liaison/API	Intégré - Fig. 9
04	Journal de détection temps réel	Décisions d'inférence et de politique d'alerte	Intégré - Fig. 10
05	Alerte temps réel persistée	Persistance d'alerte de bout en bout	Intégré - Fig. 14
06	Page de connexion	Point d'entrée humain protégé par JWT	Intégré - Fig. 11
07	Tableau de bord administrateur	Supervision opérationnelle et interface administrateur	Intégré - Fig. 12
08	Tableau de bord Lecteur/RBAC	Interface en lecture seule selon le rôle	Capturé localement ; non intégré
09	Détail d'alertes	Inspection d'un événement stocké	Intégré - Fig. 13
10	Journaux d'audit	Traçabilité des événements de sécurité	Intégré - Fig. 15
11	Pile Docker Compose saine	Déploiement reproductible à 3 services	Capturé localement ; non intégré
12	Santé API + en-têtes	Disponibilité en exécution et durcissement HTTP	Intégré - Fig. 16
13	Succès GitHub Actions	CI automatisée backend/frontend/Docker	Intégré - Fig. 4
14	Résultats du modèle de base	Évaluation de la référence paquet	Intégré - Fig. 6

#	Preuve	Ce que cela prouve	Statut dans ce rapport
15	Résultats du modèle comportemental	Évaluation à cinq classes au niveau session	Intégré - Fig. 7

E.1 Sources et jalons du projet

Ce rapport final consolide les jalons validés produits durant août 2026 et intègre les preuves finales de livraison capturées le 29 août 2026. Il distingue l'évaluation contrôlée en laboratoire du comportement opérationnel en direct, documente l'écart d'étalonnage de confiance observé pendant les tests SYN flood en direct et évite de présenter les métriques de laboratoire comme des garanties de performance dans un environnement réel arbitraire.